

PORTADA

Arquitectura de Software

Semana 3 — Integración de UML para Vistas Arquitectónicas

Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

4 vistas
1 solo sistema



CASO · FICHA TÉCNICA

El modelo 4+1 de Kruchten, 1995

Autor	Philippe Kruchten, en Rational Software (hoy parte de IBM)
Publicación	"Architectural Blueprints — The '4+1' View Model of Software Architecture", revista <i>IEEE Software</i> , 1995
Contexto	Kruchten trabajaba en el Rational Unified Process (RUP), un proceso de desarrollo usado en proyectos grandes, incluyendo defensa y aeroespacial
Problema que resolvía	Ningún diagrama único lograba describir un sistema completo sin volverse ilegible
Legado	Influyó directamente en la organización de UML y sigue siendo referencia estándar para documentar arquitectura hoy

CASO · EL PROBLEMA

Un solo diagrama no alcanza

Antes de Kruchten, era común intentar capturar toda la arquitectura de un sistema en **un único diagrama gigante**: clases, servidores, procesos y flujos de usuario, todo mezclado.

- El resultado era ilegible para cualquier audiencia específica.
- Un gerente de proyecto no necesita ver las clases; un administrador de infraestructura no necesita ver la lógica interna de cada clase.
- **Distintas personas necesitan distintas preguntas respondidas** sobre el mismo sistema.

CASO · LA SOLUCIÓN

Cuatro vistas, más los escenarios que las unen

Kruchten propuso describir la arquitectura con **4 vistas independientes**, cada una respondiendo una pregunta distinta, más un quinto elemento —los **escenarios (+1)**— que muestra cómo las cuatro vistas trabajan juntas ante un caso de uso concreto.

De ahí el nombre: "4+1" — cuatro vistas estructurales, más una vista de escenarios que las conecta.

TRANSICIÓN

De la idea de Kruchten a las herramientas de hoy

El modelo 4+1 no es un formato de dibujo específico — es una **forma de organizar qué documentar**. La notación que se volvió estándar para dibujar esas vistas es **UML** (Unified Modeling Language).

Hoy aprendemos UML no como fin en sí mismo, sino como la herramienta para producir las vistas arquitectónicas que se entregan en el Sprint 1 de este curso.

CONCEPTO

¿Qué es UML?

UML (Unified Modeling Language — Lenguaje Unificado de Modelado) es una notación gráfica estándar, mantenida por el *Object Management Group* (OMG), para representar la estructura y el comportamiento de un sistema de software mediante diagramas.

No es un lenguaje de programación: no se ejecuta. Es un lenguaje para **comunicar decisiones de diseño y arquitectura** entre personas, antes, durante y después de escribir código.

CONCEPTO · VISTA 1

Vista lógica

Responde: **¿qué funcionalidad ofrece el sistema y cómo se organiza conceptualmente?**

- Se documenta con **diagramas de clases** y **diagramas de paquetes**.
- Muestra las entidades del dominio y cómo se agrupan en módulos lógicos.
- Es la vista que más le importa a quien diseña la lógica de negocio.

El diagrama de paquetes del Sprint 1 de este curso corresponde a esta vista.

CONCEPTO · VISTA 2

Vista de desarrollo

Responde: **¿cómo se organiza el código en componentes, módulos y sus interfaces?**

- Se documenta con **diagramas de componentes**, mostrando cada pieza y sus interfaces expuestas.
- Le importa directamente al equipo de desarrollo: qué construye cada quién, y qué contrato debe cumplir cada componente frente a los demás.

El diagrama de componentes con interfaces del Sprint 1 corresponde a esta vista.



CONCEPTO · VISTA 3

Vista de procesos

Responde: **¿qué pasa en tiempo de ejecución? ¿qué procesos se comunican, en qué orden, con qué concurrencia?**

- Se documenta con **diagramas de secuencia** y **diagramas de actividad**.
- Es clave cuando hay concurrencia, procesos paralelos o sincronización entre componentes — como los 4.000 carritos del caso de Denver, visto el martes.

CONCEPTO · VISTA 4

Vista física (de despliegue)

Responde: ¿en qué hardware o infraestructura corre cada parte del sistema?

- Se documenta con **diagramas de despliegue**: servidores, nodos, dispositivos y cómo se conectan.
- Le importa a quien administra infraestructura, redes y operaciones.

El diagrama de despliegue completo es un entregable del Sprint 2 de este curso, más adelante en el semestre.



CONCEPTO · +1

Escenarios: la vista que conecta a las demás

El "+1" son los **casos de uso o escenarios** más significativos del sistema: describen una situación real (ej. "un usuario se registra") y muestran cómo participan simultáneamente las cuatro vistas anteriores para resolverla.

Sirve como validación: si un escenario importante no se puede explicar recorriendo las cuatro vistas, algo falta en la arquitectura documentada.



CONTRAEJEMPLO

Mito: "UML es solo el diagrama de clases"

Es común reducir UML a un único diagrama —normalmente el de clases— porque es el más enseñado. **UML incluye más de una decena de tipos de diagrama**, cada uno útil para una vista distinta del modelo 4+1: clases y paquetes (lógica), componentes (desarrollo), secuencia y actividad (procesos), despliegue (física), y casos de uso (escenarios).

Usar solo el diagrama de clases para "documentar la arquitectura" es como describir un edificio solo con el plano eléctrico.

TENSIÓN CONCEPTUAL

¿Hay que modelar todo en UML antes de programar?

Algunos equipos insisten en documentar exhaustivamente las cuatro vistas antes de escribir una sola línea de código de negocio. Otros consideran que eso retrasa la entrega sin garantizar que el modelo sea correcto — y prefieren programar primero y diagramar solo lo esencial, después.

¿Cuánto UML es "suficiente" antes de empezar a programar el Sprint 1 de este curso?



INTERACCIÓN

¿Qué vista responde esta pregunta?

Para cada pregunta, identifiquen a cuál de las cuatro vistas (o al "+1") corresponde:

1. ¿Cuántos servidores necesita el sistema y cómo están conectados en red?
2. ¿Qué módulos existen y cómo se agrupan las clases del dominio?
3. ¿Qué pasa, paso a paso, cuando dos procesos intentan escribir el mismo dato al mismo tiempo?
4. ¿Qué componente expone la función de "generar factura" y con qué interfaz?

5 minutos · respondan en voz alta o en el chat

SÍNTESIS

Ideas clave de hoy

1. El modelo **4+1 de Kruchten (1995)** nació porque un solo diagrama no puede describir un sistema completo sin volverse ilegible.
2. Las cuatro vistas son: **lógica, de desarrollo, de procesos y física**; el "+1" son los escenarios que las conectan.
3. **UML** es la notación estándar para dibujar esas vistas — no un lenguaje de programación, sino de comunicación.
4. El Sprint 1 de este curso exige exactamente dos de esas vistas: **diagrama de paquetes** (vista lógica) y **diagrama de componentes con interfaces** (vista de desarrollo).
5. No hay consenso sobre cuánto modelar antes de programar — depende del riesgo y la complejidad del sistema.

INSTRUCCIONES

Taller de hoy — Implementación de ejemplo

Demostración guiada por el docente — 30 a 40 minutos

El docente modela en vivo, sobre el mismo backend Spring Boot usado en sesiones anteriores, el **diagrama de paquetes** y el **diagrama de componentes con interfaces** correspondientes a ese código — exactamente los dos entregables de vista arquitectónica del Sprint 1.

Leyendo la vista de desarrollo en el ejemplo

Elemento del código	Elemento del diagrama de componentes
SaludoController	Componente "Controlador", expone la interfaz REST <code>/api/saludo</code>
SaludoService	Componente "Servicio", expone la interfaz que consume el Controlador
SaludoRepository	Componente "Repositorio", expone la interfaz que consume el Servicio

Cada flecha entre componentes en el diagrama corresponde exactamente a una dependencia real en el código

(§ `@Autowired`) — el diagrama no es una idealización, es un reflejo fiel de la estructura.

No hay entregable de estudiantes en esta sesión: el objetivo es poder replicar este mismo ejercicio sobre el código propio del Sprint 1.

CIERRE

Para la próxima semana

- Practicar dibujando el diagrama de paquetes y de componentes de un proyecto propio pequeño, aunque sea informalmente.
- Traer identificados los módulos principales que tendrá el proyecto de Fábrica Escuela.

La Semana 4 aborda el rol del arquitecto de software y los Requisitos No Funcionales bajo la norma ISO 25010 — el siguiente paso después de saber documentar una arquitectura.

